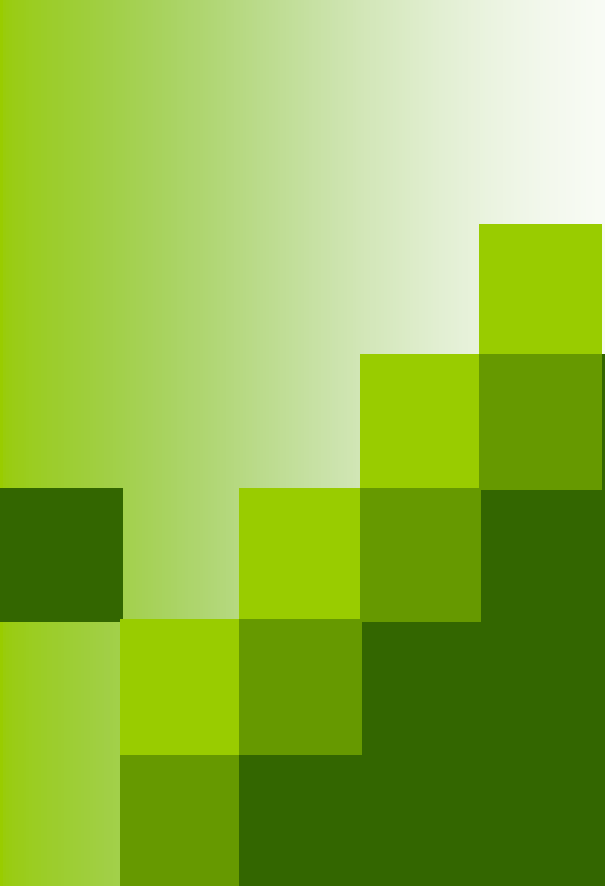




Диаграммы UML

Диаграмма классов
(*Class Diagram*)



Основные вопросы

- Что такое диаграмма классов
- Компоненты диаграммы классов и их назначение
- Пример диаграммы классов
- Расширение языка UML для построения моделей программного обеспечения и бизнес-систем



Диаграмма классов

- Является центральным звеном объектно-ориентированного подхода
- Содержит информацию об объектах системы и статических связях между объектами
- Отражает *декларативные знания* о предметной области
- Оперирует понятиями *класса, объекта, отношения, пакета*

Класс

- **Класс** – это множество объектов, которые обладают *одинаковой* структурой, поведением и отношениями с объектами из других классов.

Имя_класса

← Простейший вид класса состоит только из секции имени

Имя_класса

атрибуты класса

← Класс с указанием атрибутов (переменных)

Полное описание класса, состоящее из 3 разделов (секций) – секции имени, секции атрибутов, секции операций

Имя_класса

атрибуты класса

операции класса()

Класс

- **Имя класса** должно быть уникально
- Имя класса должно начинаться с заглавной буквы.
- Класс может не иметь экземпляров или объектов. В этом случае он называется **абстрактным классом**, а для обозначения его имени используется *курсив*

Атрибуты класса

- **Атрибут = свойство**, которое является общим для всех объектов данного класса
- Общий формат записи атрибутов:
*<квантор видимости> <имя атрибута>
[кратность]: <тип атрибута> =
<исходное значение> {строка-
свойство}*

Атрибуты класса.

Квантор видимости

- Квантор видимости может принимать одно из следующих значений: **+**, **#**, **-**, **~**.
- «**+**» - атрибут с областью видимости типа ***общедоступный*** (public).
- «**#**» - атрибут с областью видимости типа ***защищенный*** (protected).
- «**-**» - атрибут с областью видимости типа ***закрытый*** (private).
- «**~**» - атрибут с областью видимости типа ***пакетный*** (package).

Атрибуты класса.

Имя атрибута

- Представлено в виде *уникальной* строки текста
- Имя атрибута является единственным обязательным элементом в синтаксическом обозначении атрибута
- Должно начинаться со строчной буквы
- По практическим соображениям записывается *без пробелов*

Атрибуты класса.

Кратность атрибута

- ***Кратность атрибута*** характеризует общее количество конкретных атрибутов данного типа, входящих в состав отдельного класса.
- **Формат**: *[нижняя граница . . верхняя граница]*
- ***Примеры: [0..1], [0..*], [1..3,5..7]***

Атрибуты класса. Тип атрибута

- Выражение, определяемое некоторым типом данных (например, в зависимости от языка программирования)
- В простейшем случае – *осмысленная строка текста*.
- Пример:
цвет: Color
имяСотрудника[1..2]: String;
видимость: Boolean

Атрибуты класса.

Исходное значение

- Служит для задания некоторого начального значения в момент *создания* отдельного экземпляра класса

- Пример:

цвет: Color = (255, 0, 0)

*имяСотрудника[1..2]: String = 'Иван
Иванов';*

видимость: Boolean = истина

Атрибуты класса.

Строка-свойство

- Служит для указания **дополнительных свойств атрибута**, которые могут характеризовать особенности изменения значений атрибута в ходе выполнения соответствующей программы.
- Это значение принимается за **исходное значение атрибута**, которое не может быть изменено в дальнейшем.
- Пример:
заработнаяПлата: Currency = \$500 {frozen}

Операции класса

- Представляют собой некоторый сервис, который предоставляет каждый экземпляр класса или объект по требованию своих клиентов.
- Правила записи операций:
*<квантор видимости> <имя операции>
(список параметров): <выражение
типа возвращаемого значения>
{строка-свойство}*

Операции класса.

Список параметров

- *Список параметров* является перечнем разделенных запятой формальных параметров, каждый из которых, в свою очередь, может быть представлен в следующем виде:

*<вид параметра> <имя параметра> :
<выражение типа> = <значение
параметра по умолчанию>*

Операции класса.

Строка-свойство

- **Строка-свойство** служит для указания значений свойств, которые могут быть применены к данной операции.
- Например, для указания последовательности действий будет использована строка-свойство вида:

{concurrency = имя} ,

где *ИМЯ* может принимать одно из следующих значений:

- **sequential** (последовательная),
- **concurrent** (параллельная),
- **guarded** (охраняемая)

Операции класса. Примеры

- +нарисовать (форма : Многоугольник = прямоугольник, цветЗаливки : Color = (0, 0, 255));
- -изменитьСчетКлиента (номерСчета : Integer) : Currency;
- #выдатьСообщение() : ('Ошибка деления на ноль').



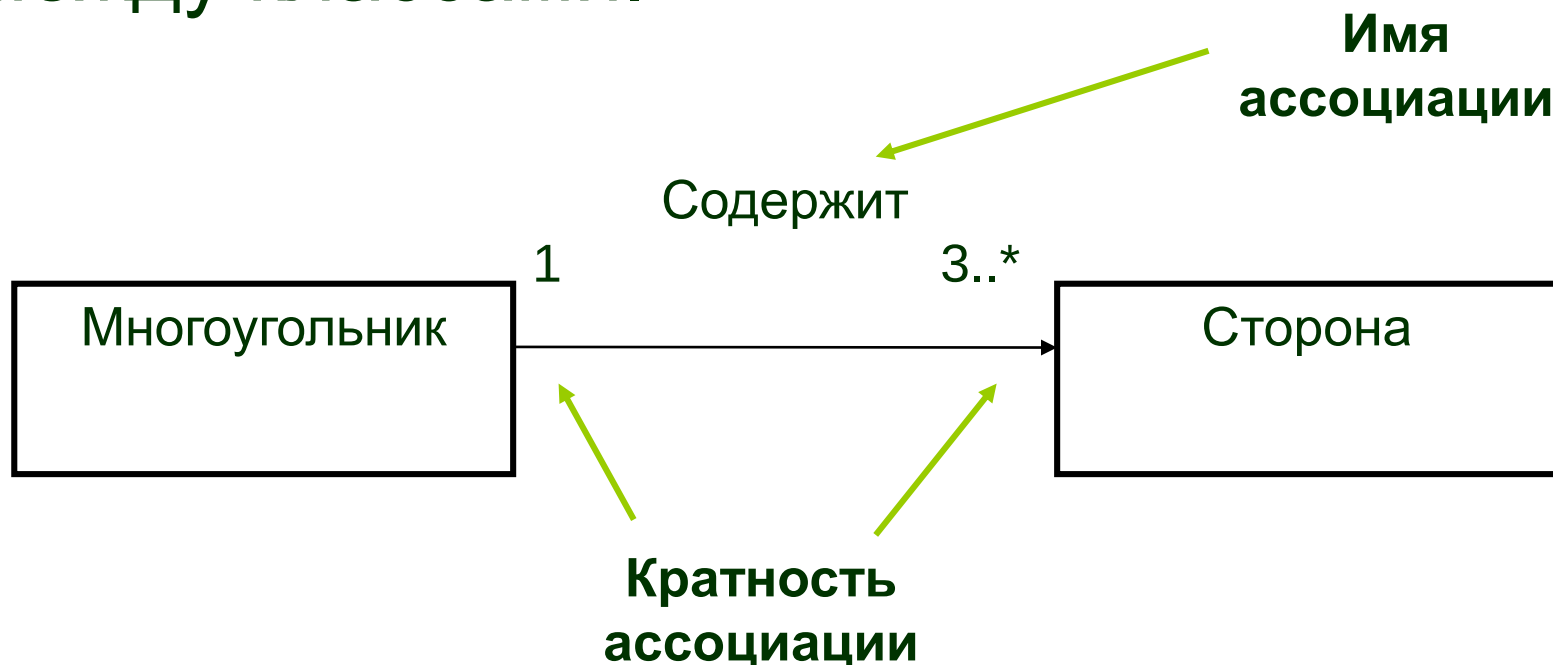
Отношения между классами

Базовыми отношениями на диаграмме классов являются:

- отношения **ассоциации** (*association*);
- отношения **обобщения** (*generalization*);
- отношения **агрегации** (*aggregation*);
- отношения **композиции** (*composition*);
- отношения **зависимости** (*dependency*).

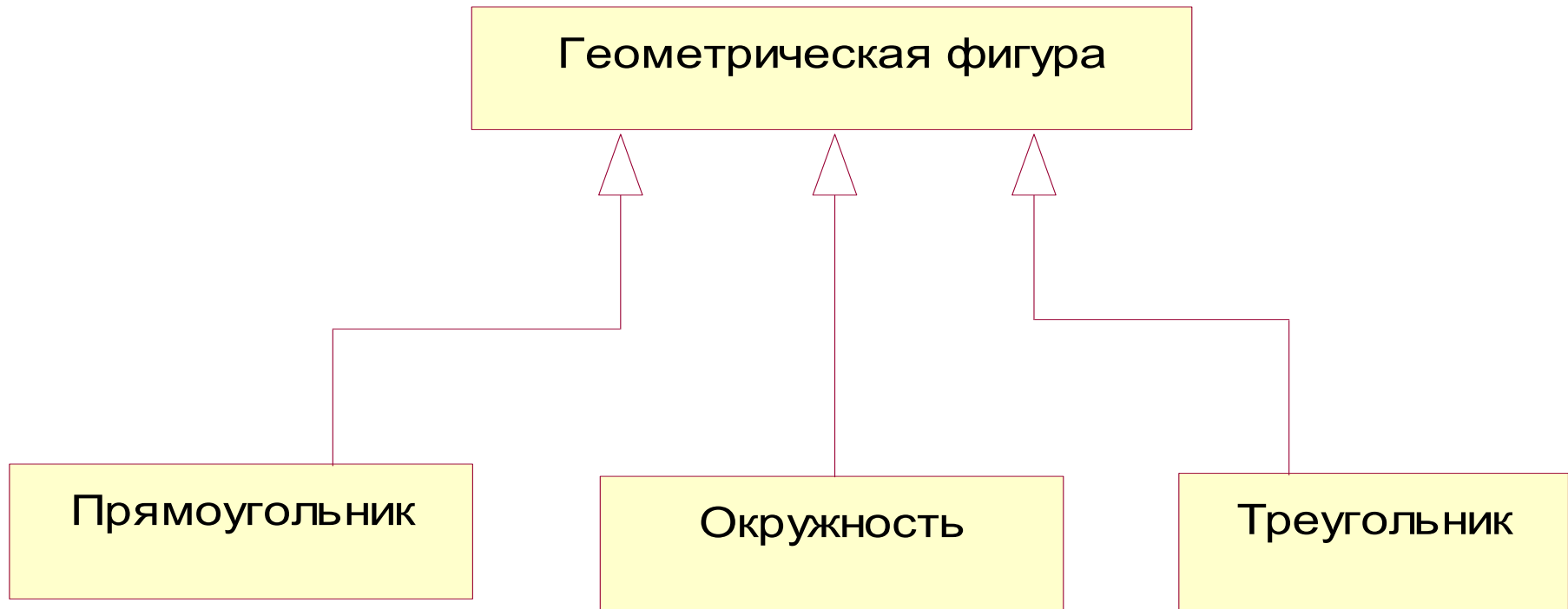
Отношение ассоциации

- **Отношение ассоциации** свидетельствует о наличии произвольного отношения между классами.



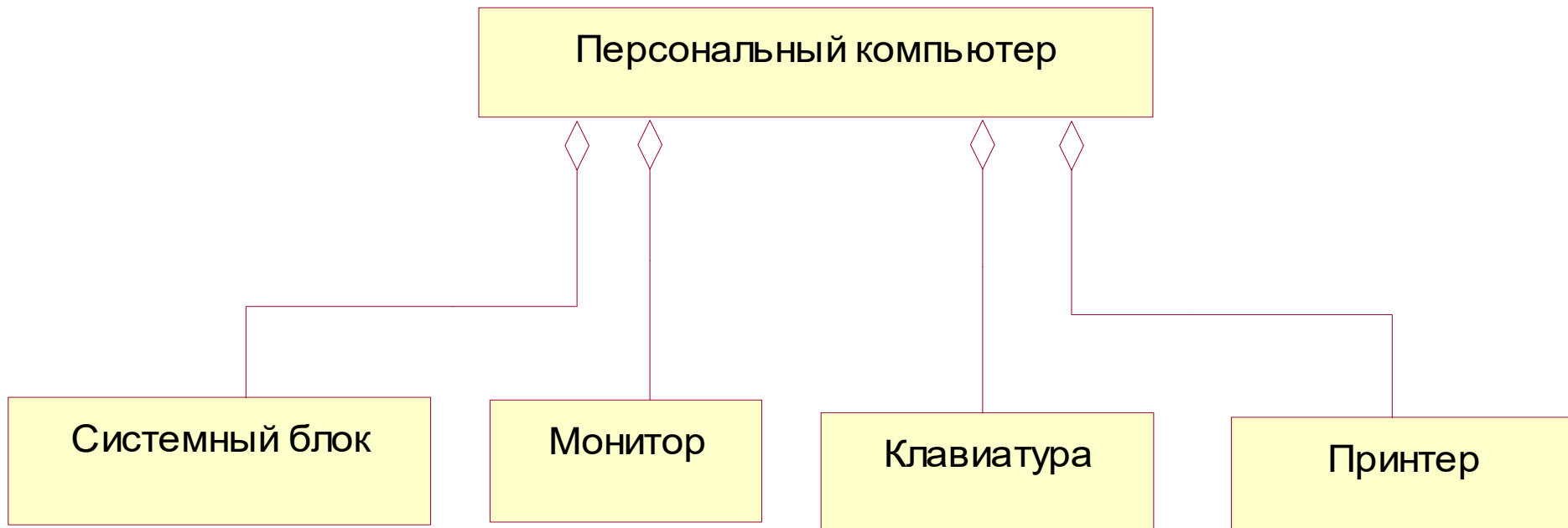
Отношение обобщения

- Является отношением *классификации* между более общим элементом (родителем или предком) и более частным или специальным элементом (дочерним или потомком)



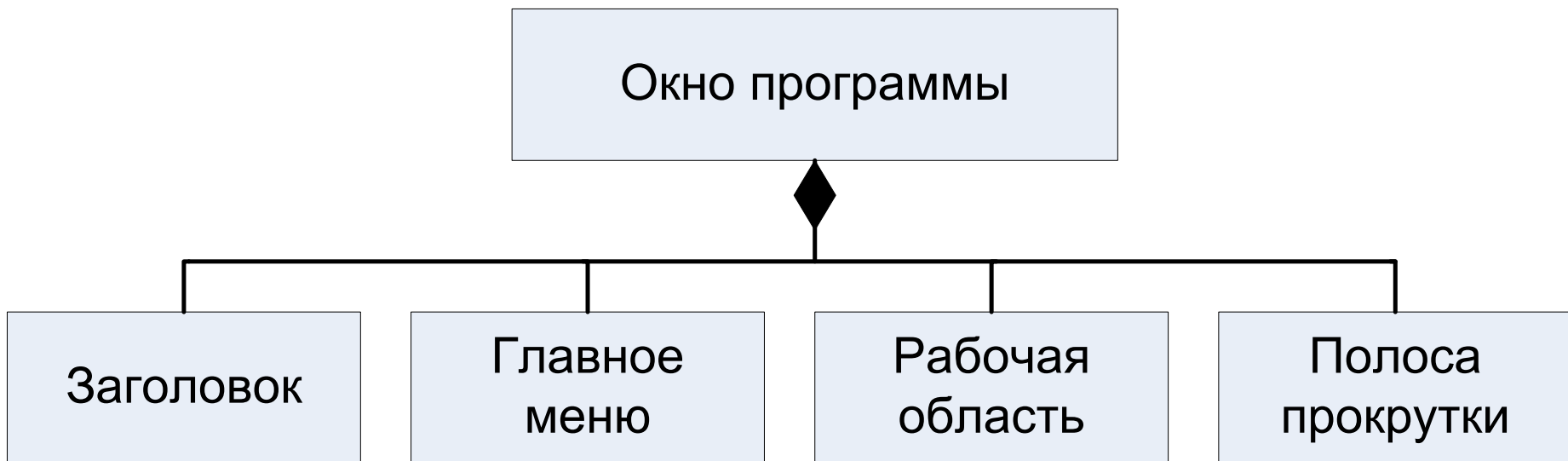
Отношение агрегации

- Смысл: один из классов представляет собой некоторую сущность, которая **включает** в себя в качестве составных частей другие сущности.
- Применяется для представления системных взаимосвязей типа «часть-целое».



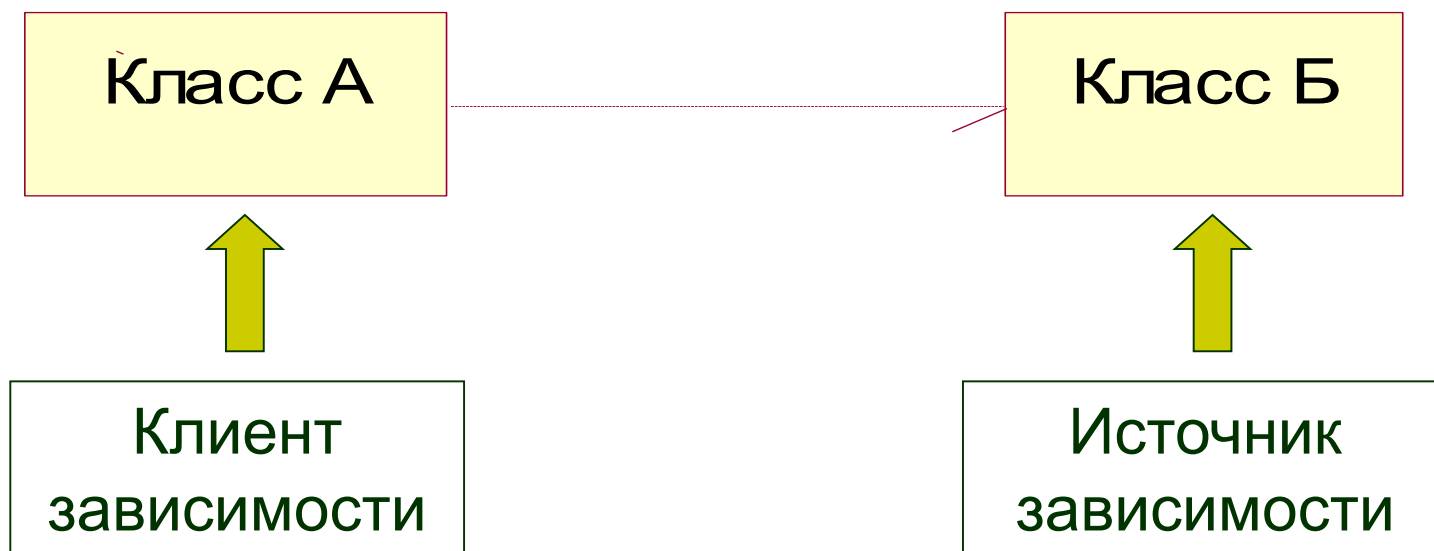
Отношение композиции

- Является частным случаем отношения агрегации.
- Части не могут выступать в отрыве от целого, т.е. с уничтожением целого уничтожаются составные части.



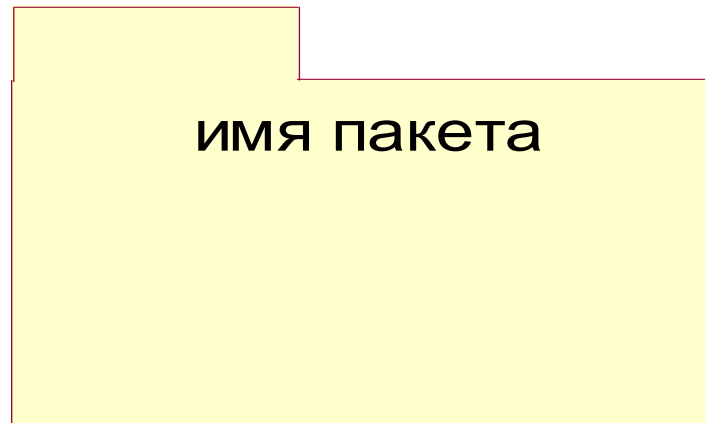
Отношение зависимости

- Используется в такой ситуации, когда некоторое изменение одного элемента модели может потребовать изменения другого элемента.

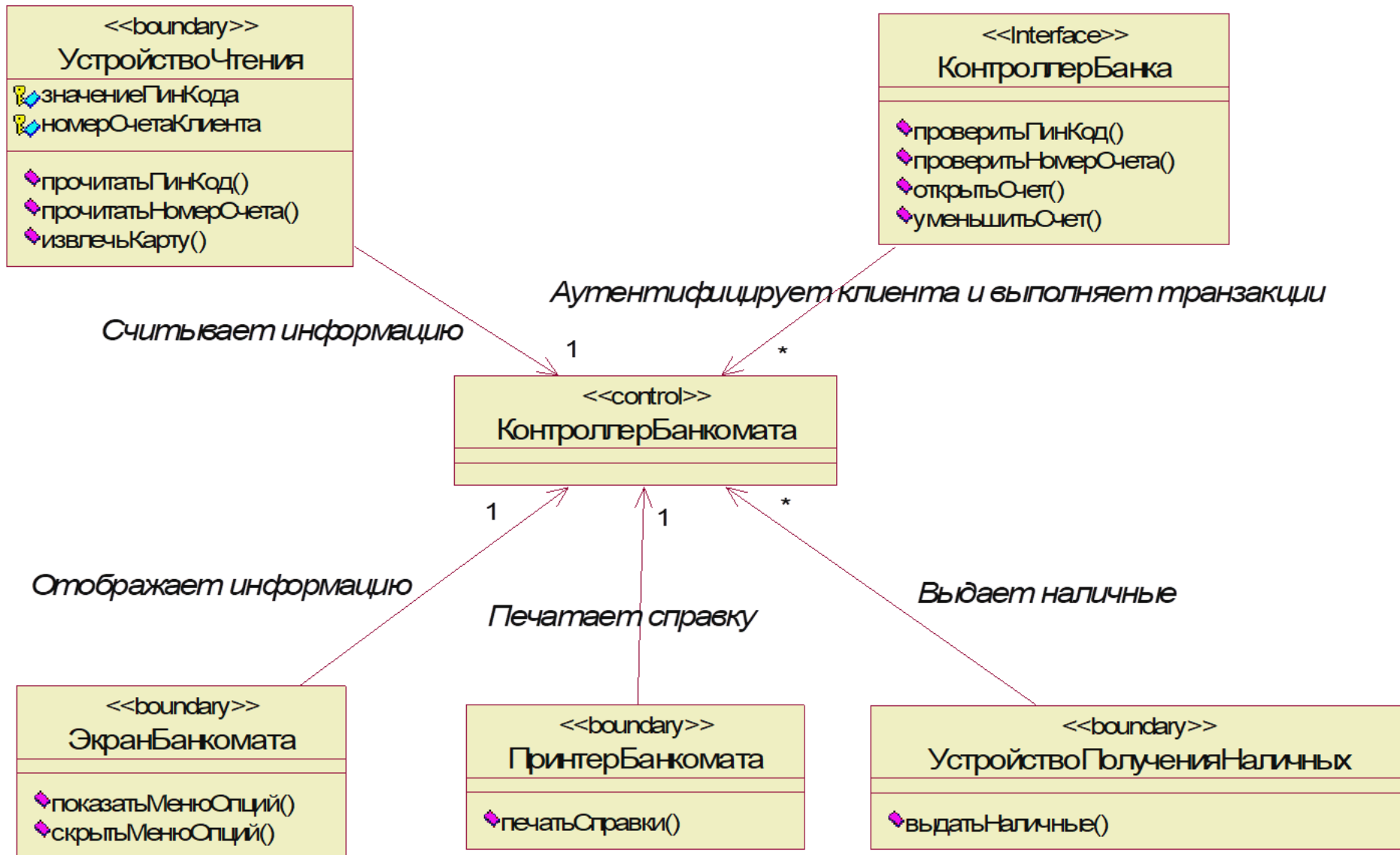


Пакеты

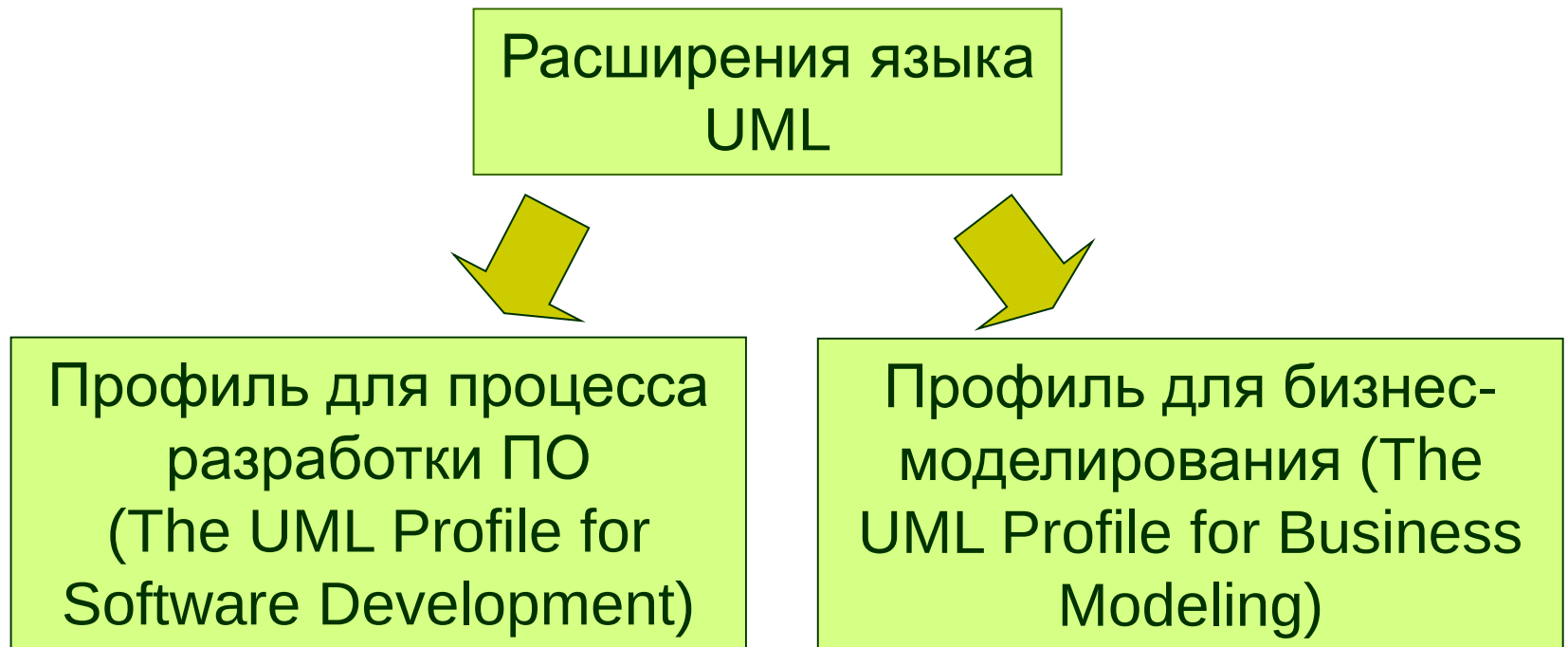
- служат для **группировки** элементов модели
- Любой пакет владеет своими элементами
- любой элемент может принадлежать *только одному пакету*



Пример диаграммы классов

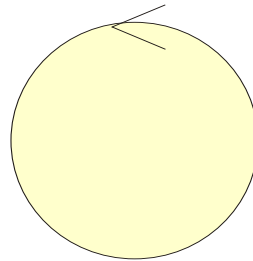


Расширения языка UML



Профиль для процесса разработки ПО

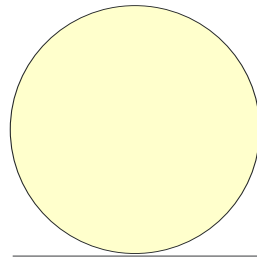
- **Управляющий класс (control)** – отвечает за координацию действий других классов.



NewClass

Профиль для процесса разработки ПО

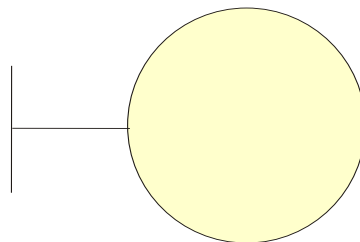
- **Класс-сущность (entity)** содержит информацию, которая должна храниться **постоянно** и не уничтожаться с уничтожением объектов данного класса или прекращением работы моделируемой системы.



NewClass2

Профиль для процесса разработки ПО

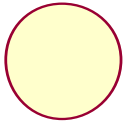
- **Граничный класс (boundary)** – располагается на границе системы с внешней средой, но является составной частью системы.



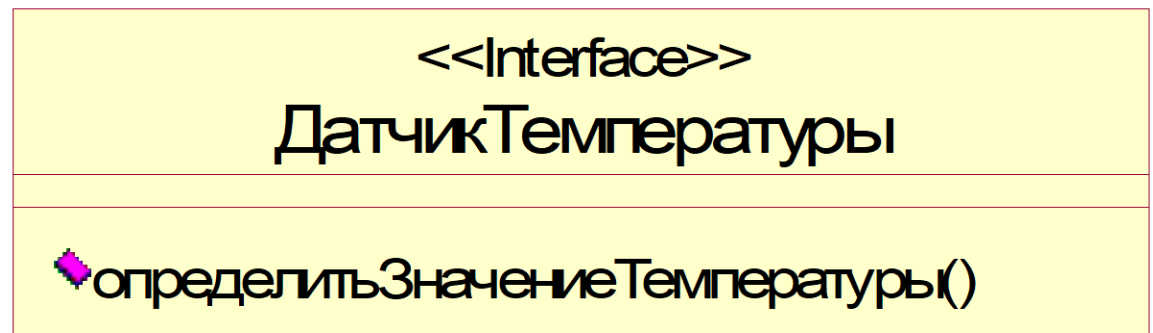
NewClass3

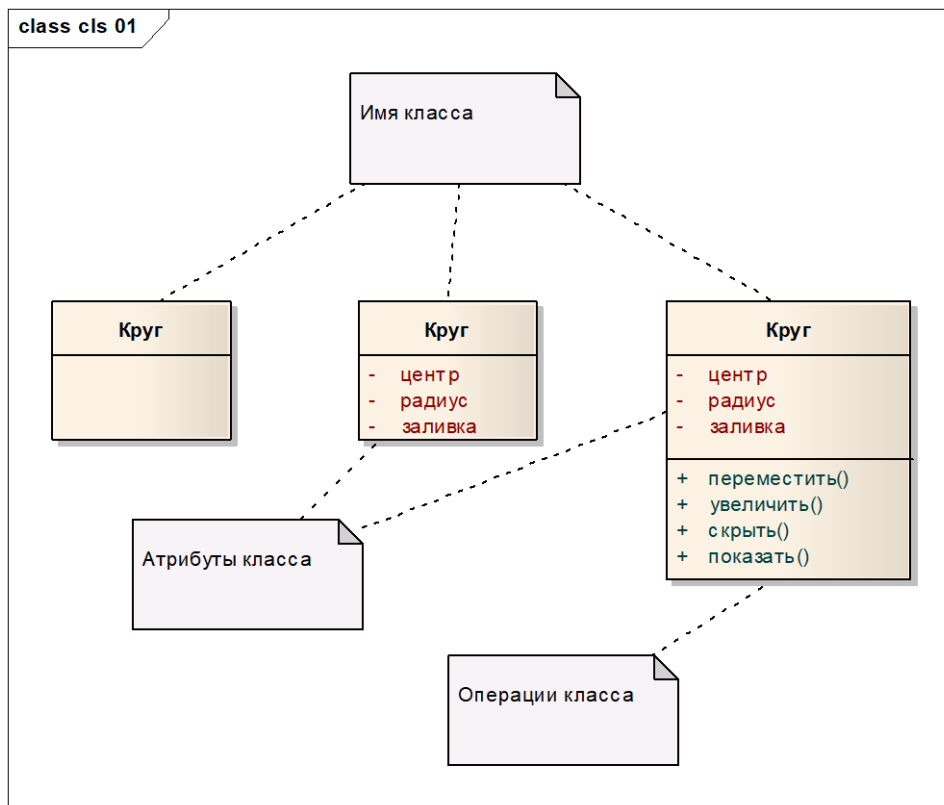
Интерфейс (interface)

- в контексте языка UML является специальным случаем класса, у которого имеются только операции и отсутствуют атрибуты.



ДатчикТем
пературы



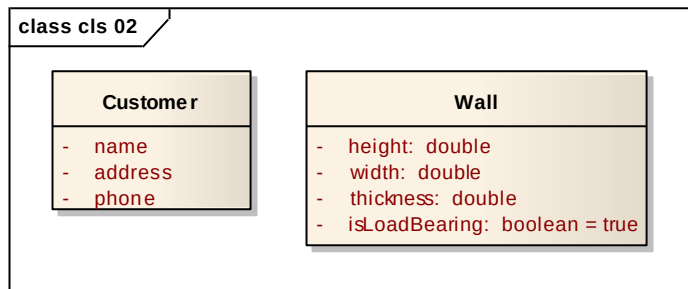


Класс (class) служит для обозначения множества объектов, которые обладают одинаковой структурой, поведением и отношениями с объектами из других классов. Графически класс изображается в виде прямоугольника, который дополнительно может быть разделен горизонтальными линиями на разделы или секции. В этих разделах могут указываться имя класса, атрибуты (переменные) и операции (методы).

- **Имя класса** должно быть уникальным в пределах проекта, который описывается некоторой совокупностью диаграмм классов (возможно, одной диаграммой). Имя класса указывается в верхней секции прямоугольника полужирным шрифтом и должно начинаться с прописной буквы.
- **Абстрактный класс** – это класс, который не может иметь экземпляров (объектов). Для обозначения его имени используется наклонный шрифт.

Атрибуты класса

Атрибут - это именованное свойство класса, общее для всех его объектов, включающее описание множества значений, которые могут принимать экземпляры этого свойства



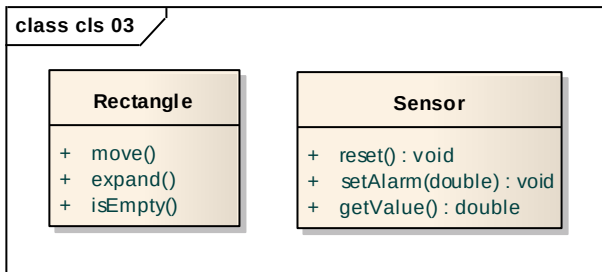
- *Класс может иметь любое число атрибутов или не иметь их вовсе*
- *Атрибут является абстракцией данных объекта или его состояния*

- *В каждый момент времени любой атрибут объекта, принадлежащего данному классу, обладает вполне определенным значением*
- *Атрибуты представлены в разделе, который расположен под именем класса*

При описании атрибута можно явным образом указывать его тип и значение, принимаемое по умолчанию

Операции класса

Операция – это реализация услуги, которую можно запросить у любого объекта класса для воздействия на поведение. Операция - это абстракция того, что может делать объект.



- *У всех объектов класса имеется общий набор операций*

- *Класс может содержать любое число операций или не содержать их вовсе*

- *Как правило (хотя не всегда) обращение к операции объекта изменяет его состояние или его данные*

- *Операции класса изображаются в разделе, расположенном ниже раздела с атрибутами. При этом можно ограничиться только именами.*

Изображая класс в UML, придерживайтесь следующих правил:

- показывайте только те свойства класса, которые важны для понимания абстракции в данном контексте
- разделяйте длинные списки атрибутов и операций на группы в соответствии с их категориями
- показывайте взаимосвязанные классы на одной и той же диаграмме

Отношения между классами

Помимо внутреннего устройства классов на диаграмме классов указываются различные отношения (связи) между ними. Совокупность типов отношений фиксирована.

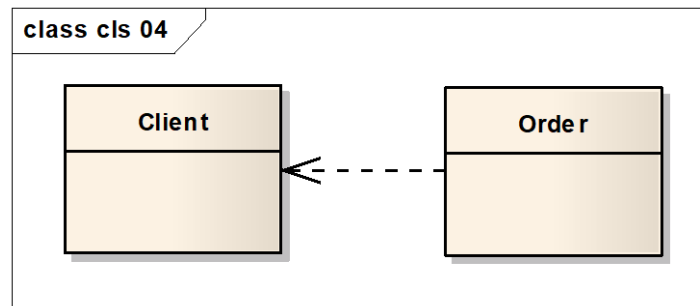
Базовые отношениями в языке UML:

- Отношение зависимости (dependency relationship)
- *Отношение ассоциации (association relationship)*
- Отношение обобщения (generalization relationship)

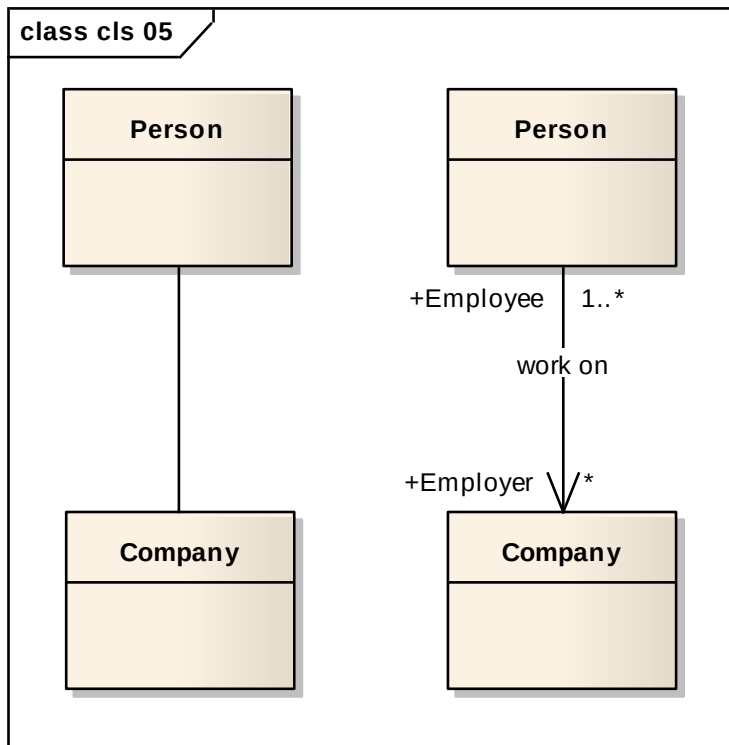
Отношение зависимости

Зависимость (dependency) - отношение использования, согласно которому изменение в спецификации одного элемента может повлиять на другой элемент, его использующий, причем обратное не обязательно.

Графически зависимость изображается пунктирной линией со стрелкой, направленной от зависимого элемента на тот, от которого он зависит.



Отношение ассоциации

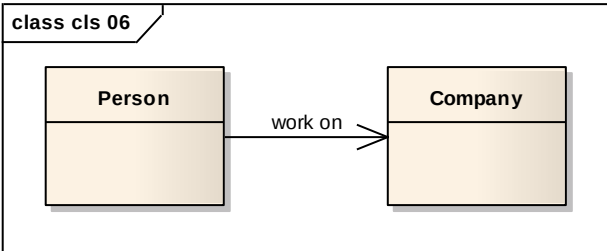


Ассоциация (association) - структурное отношение, показывающее, что объекты одного типа неким образом связаны с объектами другого типа.

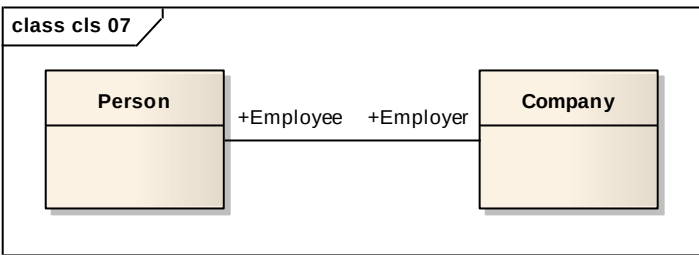
Ассоциация, связывающая два класса, называется бинарной. Можно создавать ассоциации, связывающие сразу несколько классов; они называются n-арными.

У ассоциаций могут быть: имя, роль, кратность и агрегирование.

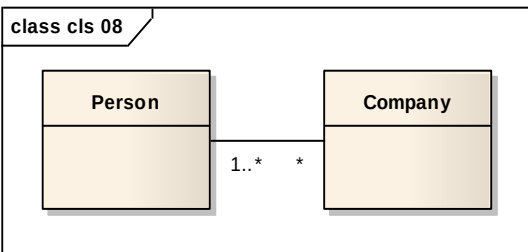
Отношение ассоциации



Имя описывает природу отношения. Чтобы избежать возможных двусмысленностей в понимании имени, указывается направление, в котором оно должно читаться.



Роль – это «лицо», которым класс, находящийся на одной стороне ассоциации, обращен к классу с другой ее стороны. Один класс может играть в разных ассоциациях как одну и ту же роль, так и различные.

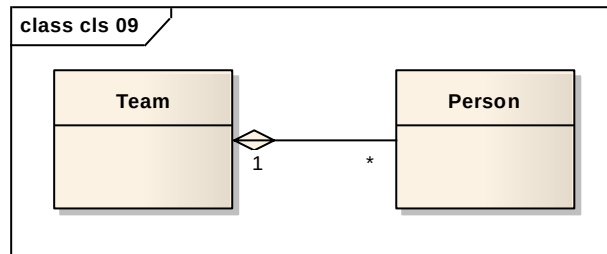


Кратность - количество объектов, которое может быть связано посредством одного экземпляра ассоциации.

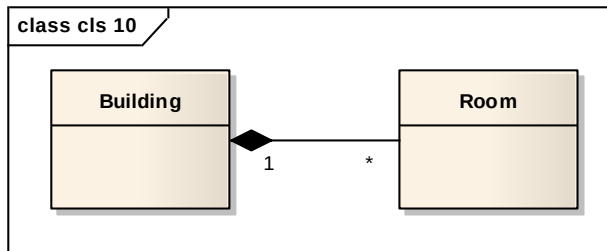
Кратность записывается либо как выражение, значением которого является диапазон значений, либо в явном виде. Кратность можно задать равной единице (1), можно указать диапазон: «ноль или единица» (0..1), «много» (0..*), «единица или больше» (1..*), «любое число объектов, кроме 2 и 5» (0..1, 3..4, 6..*).

Отношение ассоциации

Агрегирование (aggregation) показывает отношение типа «часть/целое», в котором один класс состоит из нескольких меньших классов



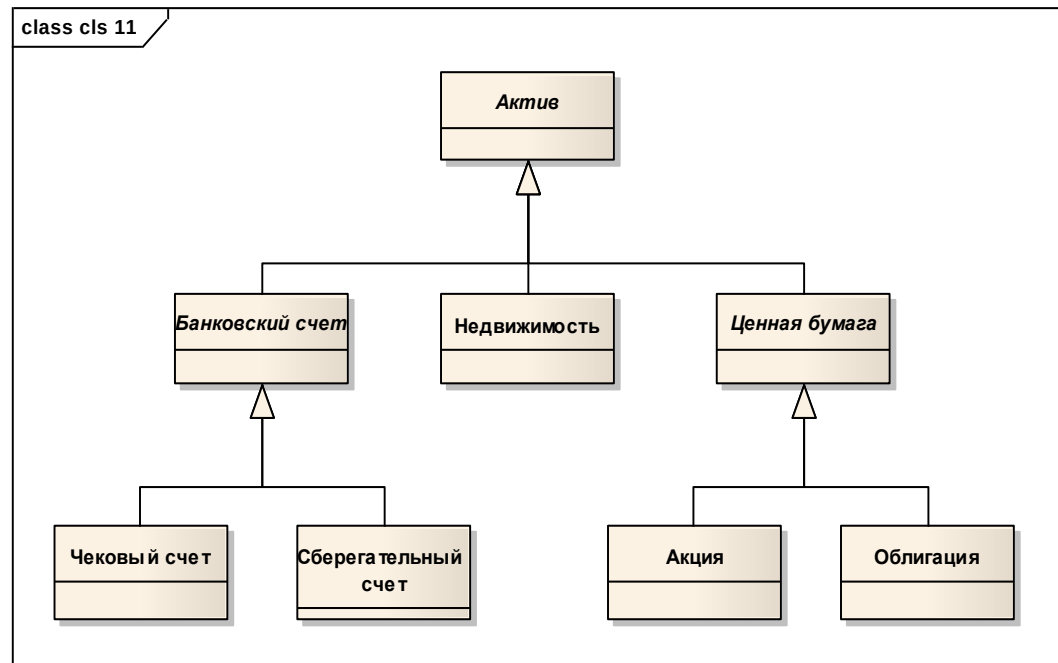
Композиция (composition) служит для выделения специальной формы отношения «часть/целое». Специфика связи заключается в том, что части не могут выступать в отрыве от целого, то есть с уничтожением целого уничтожаются и все его составные части.



Отношение обобщения

Обобщение (generalization) - это отношение между общей сущностью (суперклассом, или родителем) и ее конкретным воплощением (подклассом, или потомком).

Данное отношение описывает иерархию классов и наследование их свойств и поведения. Предполагается, что класс-потомок обладает всеми свойствами и поведением класса-предка, а также имеет свои собственные свойства и поведение, которые отсутствуют у класса-предка.

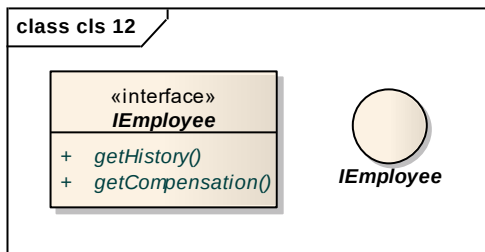


При моделировании отношений в UML соблюдайте следующие правила:

- используйте зависимость, только если моделируемое отношение не является структурным
- используйте обобщение, только если имеет место отношение типа «является»
- избегайте множественного наследования
- иерархия наследования не должна быть ни слишком глубокой (желательно не более пяти уровней), ни слишком широкой
- применяйте ассоциации прежде всего там, где между объектами существуют структурные отношения

Интерфейсы

Интерфейс (interface) - набор операций, используемый для специфицирования услуг, предоставляемых классом или компонентом.

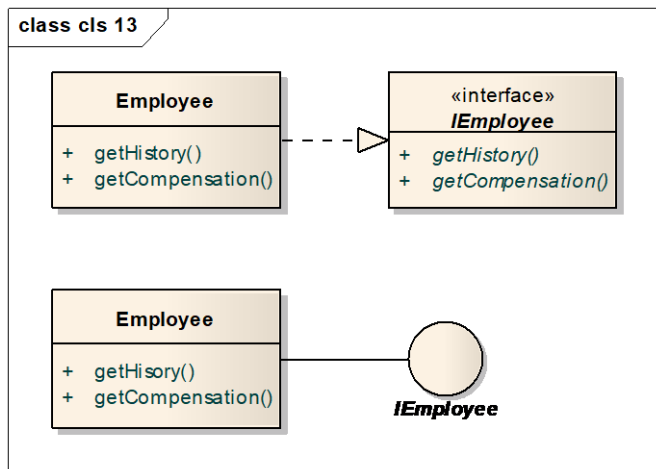


Интерфейс должен иметь уникальное имя.

Интерфейс может включать любое число операций.

Особенности:

- Интерфейсы не содержат атрибутов
- Интерфейсы не содержат реализующих операции методов

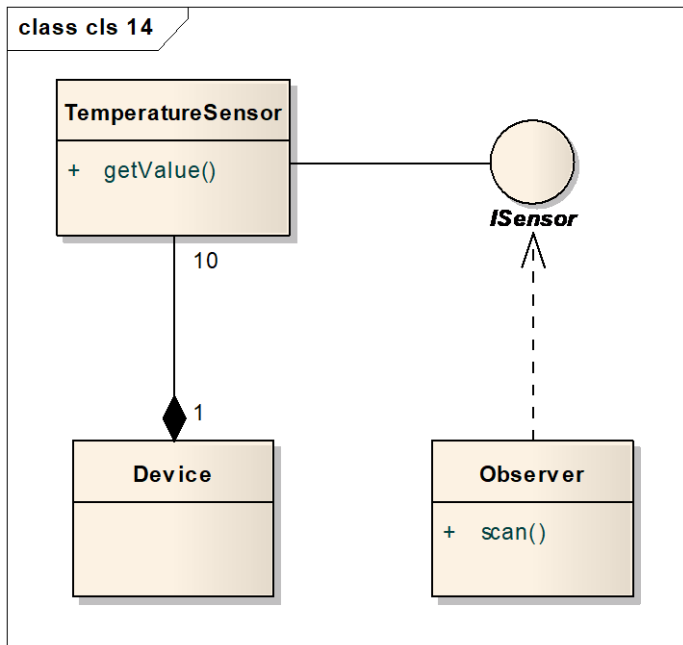


Интерфейс специфицирует контракт класса, но не накладывает никаких ограничений на свою реализацию.

Связь интерфейса с реализующим его элементом можно графически представить двумя способами:

- интерфейс представляют в виде стереотипного класса и связывают его с классом **отношением реализации**. Отношение реализации объединяет отношения обобщения и зависимости
- отношение между интерфейсом и его реализацией изображается кружочком с одной стороны класса.

Интерфейсы



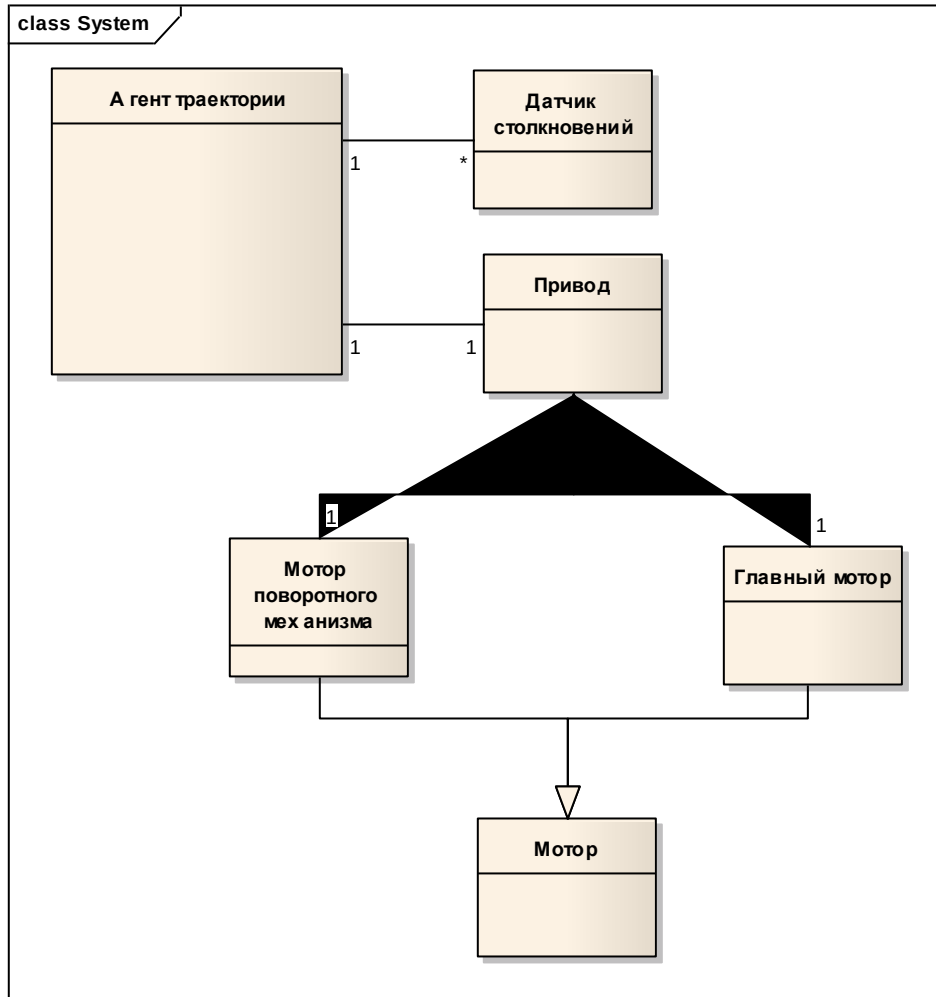
Интерфейсы могут принимать участие в отношениях обобщения, ассоциации и зависимости.

Интерфейс представляет собой стыковочный узел в системе. Он определяет условия контракта, после чего обе стороны - клиент и поставщик - могут действовать независимо друг от друга, полностью полагаясь на взаимные обязательства.

Изображая интерфейс на языке UML, руководствуйтесь приведенными ниже правилами:

- используйте сокращенную нотацию, если надо просто показать наличие стыковочного узла в системе
- используйте форму, если надо визуализировать детали самого сервиса

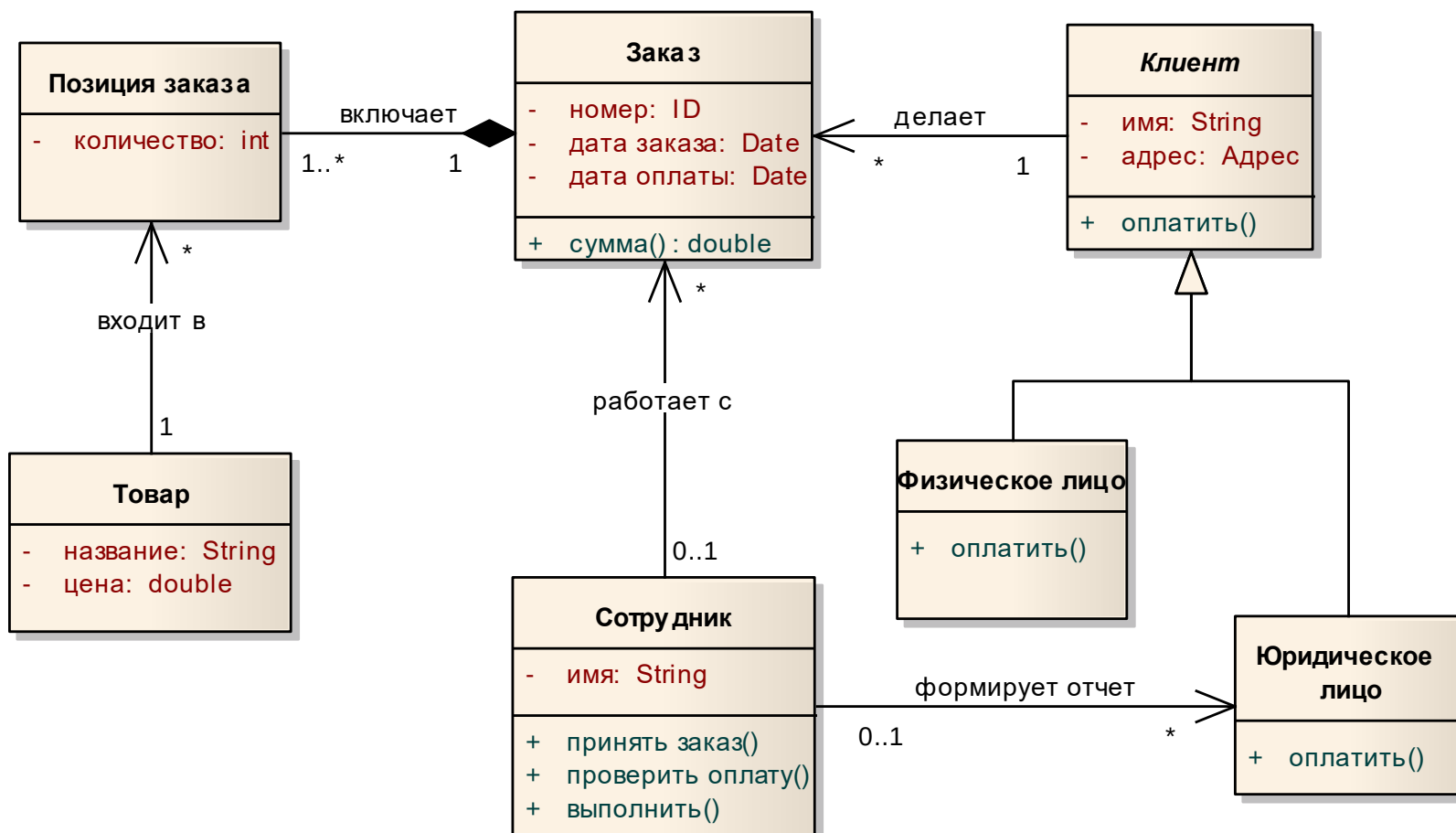
Кооперации



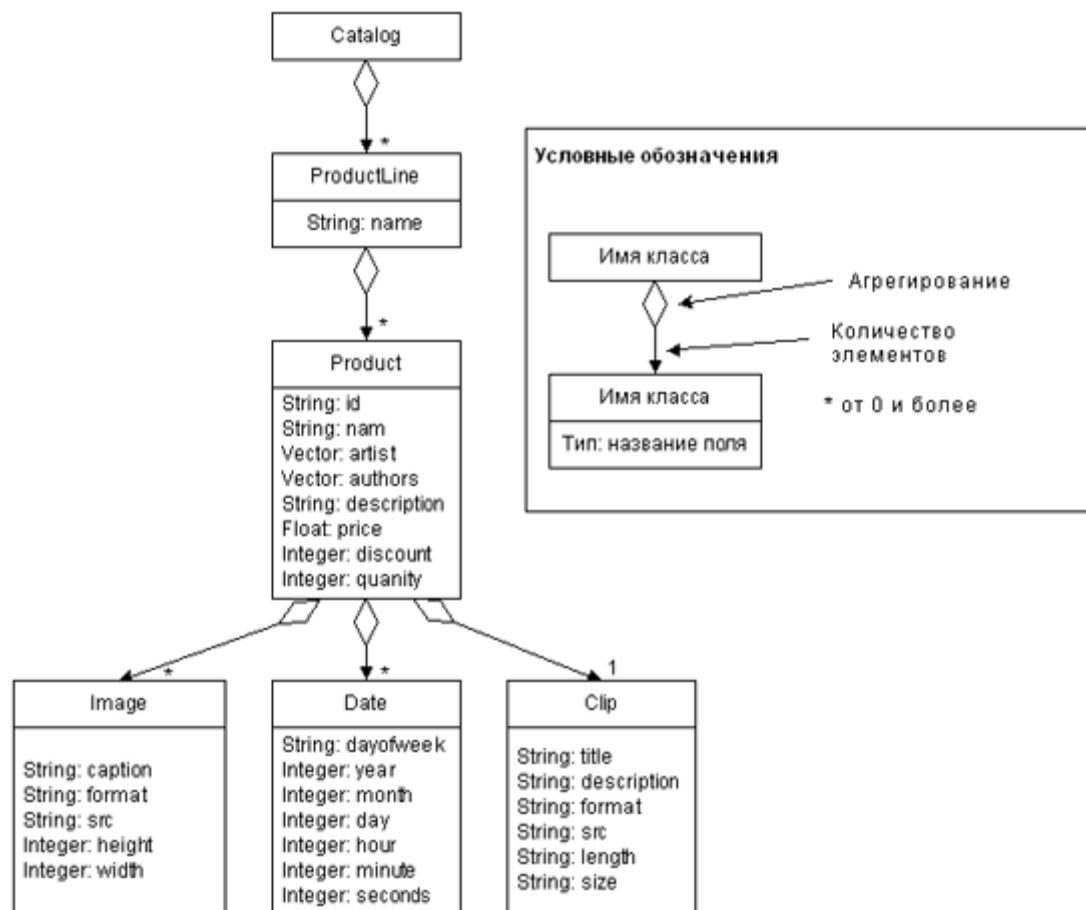
Кооперация (collaboration) - это сообщество классов, интерфейсов и других элементов, которые работают совместно для обеспечения кооперативного поведения, более значимого, чем сумма его составляющих.

Пример

class cls 16



UML



UML Jokes

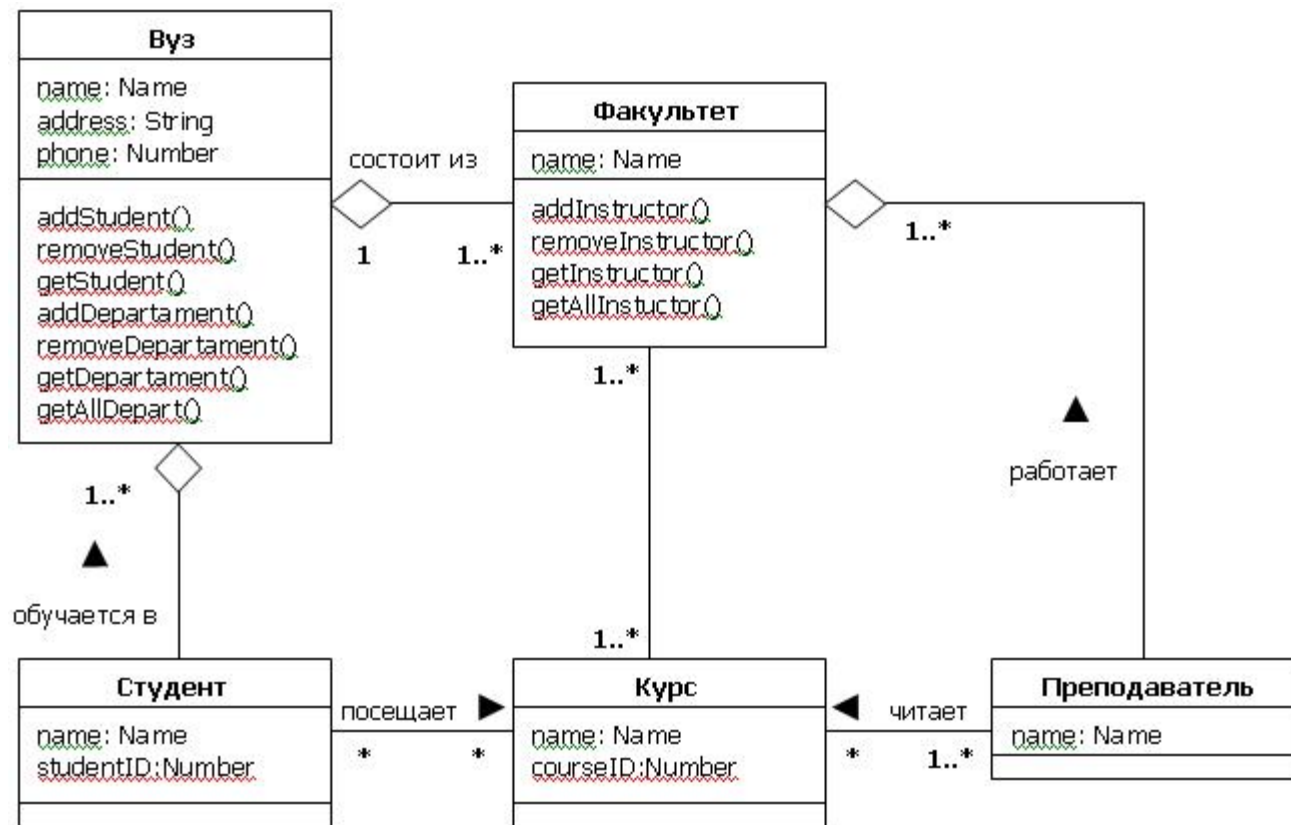


РИС.3.9